8086 Assembly Programming

Arrays, Stack, Macros & Procedures

My notes from Lab 9

CSE341: Microprocessor

BRAC University

Contents

1 A helper routine I keep reusing **2**

2 Working with arrays **3**
2.1 Summing a byte array . . . 3
2.2 Summing a word array . . . 4
2.3 Printing an array in reverse . . . 5
2.4 Copying one array into another . . . 6
2.5 Copying an array into another, reversed . . . 7
2.6 Cross-adding two arrays . . . 8
2.7 Finding the max and min . . . 10
2.8 Counting and printing the even numbers . . . 11
2.9 Counting and printing the odd numbers . . . 13
2.10 Selection sort (the home task) . . . 14

3 The stack **17**
3.1 Basic push and pop . . . 17
3.2 Reversing a string with the stack . . . 17
3.3 Reversing an array with the stack . . . 18
3.4 A balanced parentheses checker (for fun) . . . 19

4 Macros **21**
4.1 A factorial macro . . . 21
4.2 Reversing a string with a macro . . . 22
4.3 Max of two numbers, stretched to N numbers . . . 23
4.4 A power macro (X to the N) . . . 24
4.5 Printing all primes up to N . . . 25
4.6 Checking if a value exists in an array . . . 26

5 Procedures **28**
5.1 Displaying a string . . . 28
5.2 Reversing a string . . . 28
5.3 Max of three numbers . . . 30
5.4 Printing all primes up to N . . . 30
5.5 Checking if a value exists in an array . . . 32

6 Things worth remembering **34**
6.1 Array declaration and addressing . . . 34
6.2 Stack notes . . . 34
6.3 Macro vs. procedure . . . 34
6.4 Passing parameters . . . 34
6.5 Mistakes I've made (so you don't have to) . . . 35

1 A helper routine I keep reusing

Almost every program below eventually needs to print a number that might have more than one digit – a sum, a count, whatever. Instead of writing the same digit-extraction code again and again, I pulled it out into a procedure once and just `CALL` it from everywhere. The idea is simple: keep dividing by 10 and push each remainder onto the stack, then pop them back off. Because the stack is last-in-first-out, this naturally flips the digits back into the right printing order.

```
PRINT_NUM PROC
    ; Input : AX = unsigned number to print
    ; Destroys: AX, BX, CX, DX
    MOV BX, 10
    MOV CX, 0           ; digit counter

PN_DIVIDE:
    MOV DX, 0
    DIV BX              ; AX = AX/10, DX = remainder
    PUSH DX
    INC CX
    CMP AX, 0
    JNZ PN_DIVIDE

PN_PRINT:
    POP DX
    ADD DL, 30H
    MOV AH, 2
    INT 21H
    LOOP PN_PRINT
    RET
PRINT_NUM ENDP
```

Listing 1: PRINT_NUM – prints an unsigned word in AX as decimal

2 Working with arrays

2.1 Summing a byte array

```
.MODEL SMALL
.STACK 100H
.DATA
    ARR   DB 10 DUP(?)
    MSG1  DB 'Enter a digit: $'
    MSG2  DB 0DH, 0AH, 'Sum = $'
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA SI, ARR
    MOV CX, 10
INPUT_LOOP:
    LEA DX, MSG1
    MOV AH, 9
    INT 21H

    MOV AH, 1           ; read a single digit character
    INT 21H
    SUB AL, 30H         ; ASCII -> number
    MOV [SI], AL
    INC SI
    LOOP INPUT_LOOP

    ; sum the elements
    LEA SI, ARR
    MOV CX, 10
    MOV AL, 0
    MOV AH, 0
SUM_LOOP:
    ADD AL, [SI]
    INC SI
    LOOP SUM_LOOP
    MOV AH, 0           ; AX = zero-extended sum

    LEA DX, MSG2
    MOV AH, 9
    INT 21H
    CALL PRINT_NUM

    MOV AH, 4CH
    INT 21H
MAIN ENDP

; -- include PRINT_NUM procedure here (see Section 1) --

```

```
48 END MAIN
```

Listing 2: Sum of a byte array

2.2 Summing a word array

Basically the same program, except each element now takes up two bytes, so the pointer has to move by 2 instead of 1 every time.

```
1  .MODEL SMALL
2  .STACK 100H
3  .DATA
4      ARR    DW 10 DUP(?)
5      MSG1   DB 'Enter a digit: $'
6      MSG2   DB 0DH, 0AH, 'Sum = $'
7  .CODE
8  MAIN PROC
9      MOV AX, @DATA
10     MOV DS, AX
11
12     LEA SI, ARR
13     MOV CX, 10
14 INPUT_LOOP:
15     LEA DX, MSG1
16     MOV AH, 9
17     INT 21H
18
19     MOV AH, 1
20     INT 21H
21     SUB AL, 30H
22     MOV AH, 0            ; zero-extend into AX
23     MOV [SI], AX         ; store as a WORD
24     ADD SI, 2            ; word array -> advance by 2 bytes
25     LOOP INPUT_LOOP
26
27     ; sum the elements
28     LEA SI, ARR
29     MOV CX, 10
30     MOV AX, 0
31     MOV BX, 0            ; running sum
32 SUM_LOOP:
33     ADD BX, [SI]
34     ADD SI, 2
35     LOOP SUM_LOOP
36     MOV AX, BX
37
38     LEA DX, MSG2
39     MOV AH, 9
40     INT 21H
41     CALL PRINT_NUM
42
```

```
    MOV AH, 4CH
    INT 21H
MAIN ENDP

END MAIN
```

Listing 3: Sum of a word array

2.3 Printing an array in reverse

```
.MODEL SMALL
.STACK 100H
.DATA
    ARR  DB 10 DUP(?)
    MSG1 DB 'Enter a digit: $'
    MSG2 DB 0DH, 0AH, 'Reversed: $'
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA SI, ARR
    MOV CX, 10
INPUT_LOOP:
    LEA DX, MSG1
    MOV AH, 9
    INT 21H
    MOV AH, 1
    INT 21H
    SUB AL, 30H
    MOV [SI], AL
    INC SI
    LOOP INPUT_LOOP

    LEA DX, MSG2
    MOV AH, 9
    INT 21H

    ; SI currently points one PAST the last element
    DEC SI                ; point to last element
    MOV CX, 10
PRINT_REV:
    MOV DL, [SI]
    ADD DL, 30H
    MOV AH, 2
    INT 21H
    MOV DL, ' '
    INT 21H
    DEC SI
    LOOP PRINT_REV

```

```
    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 4: Reverse print of an array

2.4 Copying one array into another

```
.MODEL SMALL
.STACK 100H
.DATA
    ARR1 DB 10 DUP(?)
    ARR2 DB 10 DUP(?)
    MSG1 DB 'Enter a digit: $'
    MSG2 DB 0DH, 0AH, 'Array 2: $'
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA SI, ARR1
    MOV CX, 10
INPUT_LOOP:
    LEA DX, MSG1
    MOV AH, 9
    INT 21H
    MOV AH, 1
    INT 21H
    SUB AL, 30H
    MOV [SI], AL
    INC SI
    LOOP INPUT_LOOP

    ; copy ARR1 -> ARR2
    LEA SI, ARR1
    LEA DI, ARR2
    MOV CX, 10
COPY_LOOP:
    MOV AL, [SI]
    MOV [DI], AL
    INC SI
    INC DI
    LOOP COPY_LOOP

    ; print ARR2
    LEA DX, MSG2
    MOV AH, 9
    INT 21H
    LEA SI, ARR2
    MOV CX, 10
```

```
PRINT_LOOP:
    MOV DL, [SI]
    ADD DL, 30H
    MOV AH, 2
    INT 21H
    MOV DL, ' '
    INT 21H
    INC SI
    LOOP PRINT_LOOP

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 5: Array copy

2.5 Copying an array into another, reversed

Same as above, but this time the source pointer starts at the last element and walks backwards while the destination pointer walks forwards as usual.

```
.MODEL SMALL
.STACK 100H
.DATA
    ARR1 DB 10 DUP(?)
    ARR2 DB 10 DUP(?)
    MSG1 DB 'Enter a digit: $'
    MSG2 DB 0DH, 0AH, 'Array 2 (reversed copy): $'
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA SI, ARR1
    MOV CX, 10
INPUT_LOOP:
    LEA DX, MSG1
    MOV AH, 9
    INT 21H
    MOV AH, 1
    INT 21H
    SUB AL, 30H
    MOV [SI], AL
    INC SI
    LOOP INPUT_LOOP

    ; copy ARR1 -> ARR2 in reverse
    LEA SI, ARR1
    ADD SI, 9           ; last index of ARR1
    LEA DI, ARR2
```

```
    MOV CX, 10
COPY_LOOP:
    MOV AL, [SI]
    MOV [DI], AL
    DEC SI
    INC DI
    LOOP COPY_LOOP

    ; print ARR2
    LEA DX, MSG2
    MOV AH, 9
    INT 21H
    LEA SI, ARR2
    MOV CX, 10
PRINT_LOOP:
    MOV DL, [SI]
    ADD DL, 30H
    MOV AH, 2
    INT 21H
    MOV DL, ' '
    INT 21H
    INC SI
    LOOP PRINT_LOOP

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 6: Reverse array copy

2.6 Cross-adding two arrays

This one took me a second to wrap my head around: ARR3[i] should equal ARR1[i] + ARR2[4-i], for i = 0..4. So while one pointer moves forward through ARR1, the other has to move backward through ARR2, starting from its last element.

```
.MODEL SMALL
.STACK 100H
.DATA
    ARR1 DB 5 DUP(?)
    ARR2 DB 5 DUP(?)
    ARR3 DB 5 DUP(?)
    MSG1 DB 'Enter a digit for array 1: $'
    MSG2 DB 'Enter a digit for array 2: $'
    MSG3 DB 0DH, 0AH, 'Array 3: $'
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

```

```
    ; fill ARR1
    LEA SI, ARR1
    MOV CX, 5
FILL1:
    LEA DX, MSG1
    MOV AH, 9
    INT 21H
    MOV AH, 1
    INT 21H
    SUB AL, 30H
    MOV [SI], AL
    INC SI
    LOOP FILL1

    ; fill ARR2
    LEA SI, ARR2
    MOV CX, 5
FILL2:
    LEA DX, MSG2
    MOV AH, 9
    INT 21H
    MOV AH, 1
    INT 21H
    SUB AL, 30H
    MOV [SI], AL
    INC SI
    LOOP FILL2

    ; ARR3[i] = ARR1[i] + ARR2[4-i]
    LEA SI, ARR1
    LEA DI, ARR2
    ADD DI, 4           ; DI points to last element of ARR2
    LEA BX, ARR3
    MOV CX, 5
CROSS_ADD:
    MOV AL, [SI]
    ADD AL, [DI]
    MOV [BX], AL
    INC SI
    DEC DI
    INC BX
    LOOP CROSS_ADD

    ; print ARR3
    LEA DX, MSG3
    MOV AH, 9
    INT 21H
    LEA SI, ARR3
    MOV CX, 5
PRINT_LOOP:
    MOV DL, [SI]
```

```
    ADD DL, 30H
    MOV AH, 2
    INT 21H
    MOV DL, ' '
    INT 21H
    INC SI
    LOOP PRINT_LOOP

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 7: Cross addition of two arrays

2.7 Finding the max and min

```
.MODEL SMALL
.STACK 100H
.DATA
    ARR  DB 10 DUP(?)
    MSG1 DB 'Enter a digit: $'
    MSG2 DB 0DH, 0AH, 'Maximum: $'
    MSG3 DB 0DH, 0AH, 'Minimum: $'
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA SI, ARR
    MOV CX, 10
INPUT_LOOP:
    LEA DX, MSG1
    MOV AH, 9
    INT 21H
    MOV AH, 1
    INT 21H
    SUB AL, 30H
    MOV [SI], AL
    INC SI
    LOOP INPUT_LOOP

    LEA SI, ARR
    MOV AL, [SI]        ; assume first element is both max and
       min
    MOV AH, [SI]
    INC SI
    MOV CX, 9
FIND_LOOP:
    CMP BYTE PTR [SI], AL
    JLE CHECK_MIN
```

```
    MOV AL, [SI]          ; new max
CHECK_MIN:
    CMP BYTE PTR [SI], AH
    JGE NEXT_ELEM
    MOV AH, [SI]          ; new min
NEXT_ELEM:
    INC SI
    LOOP FIND_LOOP

    PUSH AX               ; save AL=max, AH=min

    LEA DX, MSG2
    MOV AH, 9
    INT 21H
    POP AX
    PUSH AX
    MOV DL, AL
    ADD DL, 30H
    MOV AH, 2
    INT 21H

    LEA DX, MSG3
    MOV AH, 9
    INT 21H
    POP AX
    MOV DL, AH
    ADD DL, 30H
    MOV AH, 2
    INT 21H

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 8: Max and min of an array

2.8 Counting and printing the even numbers

```
.MODEL SMALL
.STACK 100H
.DATA
    ARR   DB 10 DUP(?)
    COUNT DB 0
    MSG1  DB 'Enter a digit: $'
    MSG2  DB 0DH, 0AH, 'Even numbers: $'
    MSG3  DB 0DH, 0AH, 'Count of even numbers: $'
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX
```

```
13
14      LEA SI, ARR
15      MOV CX, 10
16  INPUT_LOOP:
17      LEA DX, MSG1
18      MOV AH, 9
19      INT 21H
20      MOV AH, 1
21      INT 21H
22      SUB AL, 30H
23      MOV [SI], AL
24      INC SI
25      LOOP INPUT_LOOP
26
27      LEA DX, MSG2
28      MOV AH, 9
29      INT 21H
30
31      LEA SI, ARR
32      MOV CX, 10
33  CHECK_LOOP:
34      MOV AL, [SI]
35      MOV AH, 0
36      MOV BL, 2
37      DIV BL                  ; AH = AL mod 2
38      CMP AH, 0
39      JNE SKIP_EVEN
40
41      MOV DL, [SI]
42      ADD DL, 30H
43      PUSH AX
44      MOV AH, 2
45      INT 21H
46      MOV DL, ' '
47      INT 21H
48      POP AX
49      INC COUNT
50  SKIP_EVEN:
51      INC SI
52      LOOP CHECK_LOOP
53
54      LEA DX, MSG3
55      MOV AH, 9
56      INT 21H
57      MOV AL, COUNT
58      MOV AH, 0
59      CALL PRINT_NUM
60
61      MOV AH, 4CH
62      INT 21H
63  MAIN ENDP
```

```
64 END MAIN
```

Listing 9: Even numbers in an array

2.9 Counting and printing the odd numbers

Mirror image of the even-number version – just flip the condition on the remainder.

```
1  .MODEL SMALL
2  .STACK 100H
3  .DATA
4      ARR   DB 10 DUP(?)
5      COUNT DB 0
6      MSG1  DB 'Enter a digit: $'
7      MSG2  DB 0DH, 0AH, 'Odd numbers: $'
8      MSG3  DB 0DH, 0AH, 'Count of odd numbers: $'
9  .CODE
10 MAIN PROC
11     MOV AX, @DATA
12     MOV DS, AX
13
14     LEA SI, ARR
15     MOV CX, 10
16 INPUT_LOOP:
17     LEA DX, MSG1
18     MOV AH, 9
19     INT 21H
20     MOV AH, 1
21     INT 21H
22     SUB AL, 30H
23     MOV [SI], AL
24     INC SI
25     LOOP INPUT_LOOP
26
27     LEA DX, MSG2
28     MOV AH, 9
29     INT 21H
30
31     LEA SI, ARR
32     MOV CX, 10
33 CHECK_LOOP:
34     MOV AL, [SI]
35     MOV AH, 0
36     MOV BL, 2
37     DIV BL
38     CMP AH, 0
39     JE SKIP_ODD
40
41     MOV DL, [SI]
42     ADD DL, 30H
43     PUSH AX
44     MOV AH, 2
```

```
    INT 21H
    MOV DL, ' '
    INT 21H
    POP AX
    INC COUNT
SKIP_ODD:
    INC SI
    LOOP CHECK_LOOP

    LEA DX, MSG3
    MOV AH, 9
    INT 21H
    MOV AL, COUNT
    MOV AH, 0
    CALL PRINT_NUM

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 10: Odd numbers in an array

2.10 Selection sort (the home task)

The idea: for each position i from 0 to n-2, scan the rest of the array for the smallest remaining value and swap it into position i. Repeat until the whole thing is sorted.

```
.MODEL SMALL
.STACK 100H
.DATA
    ARR  DB 10 DUP(?)
    MSG1 DB 'Enter a digit: $'
    MSG2 DB 0DH, 0AH, 'Sorted array: $'
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA SI, ARR
    MOV CX, 10
INPUT_LOOP:
    LEA DX, MSG1
    MOV AH, 9
    INT 21H
    MOV AH, 1
    INT 21H
    SUB AL, 30H
    MOV [SI], AL
    INC SI
    LOOP INPUT_LOOP
```

```
24
25      ; selection sort: outer index in BX (0..8)
26      MOV BX, 0
27  OUTER:
28      CMP BX, 9
29      JGE SORT_DONE
30
31      ; find index of minimum in ARR[BX..9], store index in DI
32      MOV DI, BX          ; assume current position is min
33      LEA SI, ARR
34      ADD SI, BX
35      MOV AL, [SI]        ; AL = current min value
36
37      MOV CX, BX
38      INC CX              ; inner index starts at BX+1
39  INNER:
40      CMP CX, 10
41      JGE INNER_DONE
42      LEA SI, ARR
43      ADD SI, CX
44      CMP BYTE PTR [SI], AL
45      JGE NO_NEW_MIN
46      MOV AL, [SI]        ; new minimum value
47      MOV DI, CX          ; new minimum index
48  NO_NEW_MIN:
49      INC CX
50      JMP INNER
51  INNER_DONE:
52
53      ; swap ARR[BX] and ARR[DI]
54      LEA SI, ARR
55      ADD SI, BX
56      LEA DX, ARR         ; DX temporarily used as base pointer
57      MOV AH, [SI]        ; AH = ARR[BX]
58      PUSH BX
59      MOV BX, DI
60      LEA DI, ARR
61      ADD DI, BX          ; DI -> ARR[min index]
62      POP BX
63      MOV DL, [DI]        ; DL = ARR[min index]
64      MOV [DI], AH        ; ARR[min index] = old ARR[BX]
65      MOV [SI], DL        ; ARR[BX] = old ARR[min index]
66
67      INC BX
68      JMP OUTER
69
70  SORT_DONE:
71      LEA DX, MSG2
72      MOV AH, 9
73      INT 21H
74      LEA SI, ARR
```

```
75      MOV CX, 10
76  PRINT_LOOP:
77      MOV DL, [SI]
78      ADD DL, 30H
79      MOV AH, 2
80      INT 21H
81      MOV DL, ' '
82      INT 21H
83      INC SI
84      LOOP PRINT_LOOP
85
86      MOV AH, 4CH
87      INT 21H
88  MAIN ENDP
89  END MAIN
```

Listing 11: Selection sort

3 The stack

The stack is a LIFO structure – last in, first out. `PUSH` decrements `SP` and stores a word; `POP` loads a word back and increments `SP`. Because it naturally flips the order things went in, it turns out to be a really handy tool for reversing strings and arrays without needing a second buffer.

3.1 Basic push and pop

```
.MODEL SMALL
.STACK 100H
.DATA
.CODE
MAIN PROC
    MOV AX, 5
    MOV BX, 10

    PUSH AX              ; save AX
    PUSH BX              ; save BX

    ; ... do other work that changes AX, BX ...
    MOV AX, 0
    MOV BX, 0

    POP BX               ; restore BX (LIFO: last pushed, first
       popped)
    POP AX               ; restore AX

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 12: Basic stack usage

3.2 Reversing a string with the stack

```
.MODEL SMALL
.STACK 100H
.DATA
    TEXT DB 'HELLO$'
    LEN  EQU 5
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA SI, TEXT
    MOV CX, LEN
```

```
13
14 PUSH_LOOP:
15     MOV AL, [SI]
16     MOV AH, 0
17     PUSH AX              ; push each character (as a word)
18     INC SI
19     LOOP PUSH_LOOP
20
21     LEA SI, TEXT
22     MOV CX, LEN
23 POP_LOOP:
24     POP AX               ; comes back out in reverse order
25     MOV [SI], AL
26     INC SI
27     LOOP POP_LOOP
28
29     ; TEXT now holds the reversed string; print it
30     LEA DX, TEXT
31     MOV AH, 9
32     INT 21H
33
34     MOV AH, 4CH
35     INT 21H
36 MAIN ENDP
37 END MAIN
```

Listing 13: String reversal via the stack

3.3 Reversing an array with the stack

Exact same trick, just applied to a plain numeric array instead of a string.

```
1  .MODEL SMALL
2  .STACK 100H
3  .DATA
4      ARR DB 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
5      LEN EQU 10
6  .CODE
7  MAIN PROC
8      MOV AX, @DATA
9      MOV DS, AX
10
11     LEA SI, ARR
12     MOV CX, LEN
13 PUSH_LOOP:
14     MOV AL, [SI]
15     MOV AH, 0
16     PUSH AX
17     INC SI
18     LOOP PUSH_LOOP
19
20     LEA SI, ARR
```

```
    MOV CX, LEN
POP_LOOP:
    POP AX
    MOV [SI], AL
    INC SI
    LOOP POP_LOOP

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 14: Array reversal via the stack

3.4 A balanced parentheses checker (for fun)

Wanted to try a classic stack application outside the lab tasks: push on every '(' and pop on every ')'. If a ')' shows up with nothing on the stack, or anything is left on the stack at the end, the string isn't balanced.

```
.MODEL SMALL
.STACK 100H
.DATA
    TEXT     DB '(()())$'
    MSG_OK   DB 0DH, 0AH, 'Balanced$'
    MSG_BAD  DB 0DH, 0AH, 'Not Balanced$'
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA SI, TEXT
    MOV CX, 0           ; CX tracks how many '(' are currently
       pushed

SCAN_LOOP:
    MOV AL, [SI]
    CMP AL, '$'
    JE SCAN_DONE

    CMP AL, '('
    JNE CHECK_CLOSE
    PUSH AX
    INC CX
    JMP NEXT_CHAR

CHECK_CLOSE:
    CMP AL, ')'
    JNE NEXT_CHAR
    CMP CX, 0
    JE UNBALANCED       ; closing with nothing open
```

```
    POP AX
    DEC CX

NEXT_CHAR:
    INC SI
    JMP SCAN_LOOP

SCAN_DONE:
    CMP CX, 0
    JNE UNBALANCED

    LEA DX, MSG_OK
    JMP DISPLAY

UNBALANCED:
    LEA DX, MSG_BAD

DISPLAY:
    MOV AH, 9
    INT 21H

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 15: Balanced parentheses

4 Macros

A macro gets expanded by the assembler at compile time – its body is copied inline wherever you invoke it. No `CALL`/`RET` overhead, but the code size grows every time you use it. General shape:

```
MACRO_NAME MACRO param1, param2
    ; body using param1, param2
ENDM

; usage:
MACRO_NAME arg1, arg2
```

Listing 16: Macro syntax

4.1 A factorial macro

```
FACTORIAL MACRO NUM, RESULT
    LOCAL FACT_LOOP
    PUSH AX
    PUSH CX
    PUSH DX

    MOV AX, 1            ; running product
    MOV CX, NUM
FACT_LOOP:
    CMP CX, 1
    JLE FACT_DONE
    MUL CX               ; AX = AX * CX
    DEC CX
    JMP FACT_LOOP
FACT_DONE:
    MOV RESULT, AX

    POP DX
    POP CX
    POP AX
ENDM

; --- example usage ---
.MODEL SMALL
.STACK 100H
.DATA
    N    DW 5
    ANS  DW ?
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    FACTORIAL N, ANS     ; ANS = 5! = 120
```

```
35
36      MOV AH, 4CH
37      INT 21H
38  MAIN ENDP
39  END MAIN
```

Listing 17: Factorial macro

4.2 Reversing a string with a macro

Same stack trick as before, just wrapped in a macro so it can be dropped into any program.

```
1   REVERSE_STRING MACRO STR_ADDR, LEN
2       LOCAL RS_PUSH, RS_POP
3       PUSH AX
4       PUSH CX
5       PUSH SI
6
7       LEA SI, STR_ADDR
8       MOV CX, LEN
9   RS_PUSH:
10      MOV AL, [SI]
11      MOV AH, 0
12      PUSH AX
13      INC SI
14      LOOP RS_PUSH
15
16      LEA SI, STR_ADDR
17      MOV CX, LEN
18  RS_POP:
19      POP AX
20      MOV [SI], AL
21      INC SI
22      LOOP RS_POP
23
24      POP SI
25      POP CX
26      POP AX
27  ENDM
28
29  ; --- example usage ---
30  .MODEL SMALL
31  .STACK 100H
32  .DATA
33      TEXT DB 'ASSEMBLY$'
34      LEN  EQU 8
35  .CODE
36  MAIN PROC
37      MOV AX, @DATA
38      MOV DS, AX
```

```
39
40      REVERSE_STRING TEXT, LEN
41
42      LEA DX, TEXT
43      MOV AH, 9
44      INT 21H
45
46      MOV AH, 4CH
47      INT 21H
48  MAIN ENDP
49  END MAIN
```

Listing 18: String reversal macro

4.3 Max of two numbers, stretched to N numbers

Wrote a small MAX2 macro first, then just called it repeatedly inside a loop to find the max of a whole array – keep the running max in a variable and compare it against the next element each time.

```
1   MAX2 MACRO A, B, RESULT
2       LOCAL A_IS_BIGGER, MAX_DONE
3       PUSH AX
4       MOV AX, A
5       CMP AX, B
6       JGE A_IS_BIGGER
7       MOV AX, B
8   A_IS_BIGGER:
9       MOV RESULT, AX
10      POP AX
11  ENDM
12
13  ; --- example: find max of N numbers stored in an array ---
14  .MODEL SMALL
15  .STACK 100H
16  .DATA
17      ARR  DW 3, 9, 2, 7, 5, 12, 6
18      N    EQU 7
19      MAXV DW ?
20  .CODE
21  MAIN PROC
22      MOV AX, @DATA
23      MOV DS, AX
24
25      LEA SI, ARR
26      MOV AX, [SI]          ; assume first element is the max so far
27      MOV MAXV, AX
28      ADD SI, 2
29
30      MOV CX, N
31      DEC CX                ; N-1 remaining comparisons
```

```
COMPARE_LOOP:
    MAX2 MAXV, WORD PTR [SI], MAXV
    ADD SI, 2
    LOOP COMPARE_LOOP

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 19: MAX2 macro and using it for N numbers

4.4 A power macro (X to the N)

```
POWER MACRO BASE, EXP, RESULT
    LOCAL POW_LOOP, POW_DONE
    PUSH AX
    PUSH CX

    MOV AX, 1
    MOV CX, EXP
    CMP CX, 0
    JE POW_DONE
POW_LOOP:
    IMUL WORD PTR BASE  ; AX = AX * BASE (uses DX:AX, keep EXP
        small)
    LOOP POW_LOOP
POW_DONE:
    MOV RESULT, AX

    POP CX
    POP AX
ENDM

; --- example usage: 2^5 ---
.MODEL SMALL
.STACK 100H
.DATA
    BASE_VAL DW 2
    EXP_VAL  DW 5
    RESULT   DW ?
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    POWER BASE_VAL, EXP_VAL, RESULT    ; RESULT = 32

    MOV AH, 4CH
    INT 21H
MAIN ENDP
```

```
END MAIN
```

Listing 20: Power macro

4.5 Printing all primes up to N

Plain trial division: for each candidate, try dividing by every number from 2 up to itself; if nothing divides it evenly before reaching itself, it's prime.

```
PRINT_PRIMES MACRO LIMIT
    LOCAL OUTER_LOOP, INNER_LOOP, NOT_PRIME, IS_PRIME, DONE_ALL,
       NEXT_N
    PUSH AX
    PUSH BX
    PUSH CX
    PUSH DX

    MOV BX, 2           ; candidate number, starts at 2
OUTER_LOOP:
    CMP BX, LIMIT
    JG DONE_ALL

    MOV CX, 2           ; trial divisor
INNER_LOOP:
    MOV AX, BX
    CMP CX, AX
    JGE IS_PRIME        ; divisor reached the number itself ->
       prime

    MOV DX, 0
    DIV CX              ; AX / CX, remainder in DX
    CMP DX, 0
    JE NOT_PRIME
    INC CX
    JMP INNER_LOOP

IS_PRIME:
    MOV AX, BX
    CALL PRINT_NUM
    MOV DL, ' '
    MOV AH, 2
    INT 21H

NOT_PRIME:
NEXT_N:
    INC BX
    JMP OUTER_LOOP

DONE_ALL:
    POP DX
    POP CX
```

```
    POP BX
    POP AX
ENDM

; --- example usage ---
.MODEL SMALL
.STACK 100H
.DATA
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    PRINT_PRIMES 30      ; prints 2 3 5 7 11 13 17 19 23 29

    MOV AH, 4CH
    INT 21H
MAIN ENDP

; -- include PRINT_NUM procedure here (see Section 1) --

END MAIN
```

Listing 21: Prime printing macro

4.6 Checking if a value exists in an array

```
FIND_VALUE MACRO ARR_ADDR, LEN, TARGET, FOUND_FLAG
    LOCAL SEARCH_LOOP, VALUE_FOUND, SEARCH_DONE
    PUSH AX
    PUSH CX
    PUSH SI

    MOV FOUND_FLAG, 0    ; assume not found
    LEA SI, ARR_ADDR
    MOV CX, LEN
SEARCH_LOOP:
    MOV AL, [SI]
    CMP AL, TARGET
    JE VALUE_FOUND
    INC SI
    LOOP SEARCH_LOOP
    JMP SEARCH_DONE

VALUE_FOUND:
    MOV FOUND_FLAG, 1

SEARCH_DONE:
    POP SI
    POP CX
    POP AX
```

```
ENDM

; --- example usage ---
.MODEL SMALL
.STACK 100H
.DATA
    ARR   DB 4, 8, 15, 16, 23, 42
    LEN   EQU 6
    FOUND DB ?
    MSG_YES DB 0DH, 0AH, 'Value found$'
    MSG_NO  DB 0DH, 0AH, 'Value not found$'
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    FIND_VALUE ARR, LEN, 16, FOUND

    CMP FOUND, 1
    JE PRINT_YES
    LEA DX, MSG_NO
    JMP SHOW
PRINT_YES:
    LEA DX, MSG_YES
SHOW:
    MOV AH, 9
    INT 21H

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 22: Array search macro

5 Procedures

A procedure (`PROC`/`ENDP`) is written once and called with `CALL`, which pushes the return address and jumps in; `RET` pops that address and jumps back. Unlike a macro, the code only exists once in memory no matter how many times you call it. For small programs like these, parameters are usually just passed through registers (as below); bigger programs would push them onto the stack instead.

5.1 Displaying a string

```
.MODEL SMALL
.STACK 100H
.DATA
    TEXT DB 'Hello from a procedure!$'
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA DX, TEXT        ; parameter: pointer to string in DX
    CALL DISPLAY_STRING

    MOV AH, 4CH
    INT 21H
MAIN ENDP

DISPLAY_STRING PROC
    ; Input: DX = address of a '$'-terminated string
    MOV AH, 9
    INT 21H
    RET
DISPLAY_STRING ENDP

END MAIN
```

Listing 23: Display string procedure

5.2 Reversing a string

```
.MODEL SMALL
.STACK 100H
.DATA
    TEXT DB 'PROCEDURE$'
    LEN  EQU 9
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

```

```
    LEA SI, TEXT          ; parameter: SI = string address
    MOV CX, LEN           ; parameter: CX = length
    CALL REVERSE_STRING

    LEA DX, TEXT
    MOV AH, 9
    INT 21H

    MOV AH, 4CH
    INT 21H
MAIN ENDP

REVERSE_STRING PROC
    ; Input : SI = string address, CX = length
    ; Uses the stack to reverse the characters in place
    PUSH AX
    PUSH CX
    PUSH SI
    PUSH DI

    MOV DI, SI            ; keep a copy of the start address
PUSH_LOOP:
    MOV AL, [SI]
    MOV AH, 0
    PUSH AX
    INC SI
    LOOP PUSH_LOOP

    MOV SI, DI
    POP CX                ; NOTE: length was overwritten by loop
       above,
                           ; so a caller should reload CX if reused
                              after this call
    MOV CX, LEN
POP_LOOP:
    POP AX
    MOV [SI], AL
    INC SI
    LOOP POP_LOOP

    POP DI
    POP SI
    POP CX
    POP AX
    RET
REVERSE_STRING ENDP

END MAIN
```

Listing 24: Reverse string procedure

5.3 Max of three numbers

```
.MODEL SMALL
.STACK 100H
.DATA
    A   DW 12
    B   DW 45
    C   DW 30
    MAXV DW ?
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    MOV AX, A           ; parameters passed in AX, BX, CX
    MOV BX, B
    MOV CX, C
    CALL MAX_OF_3        ; returns result in AX
    MOV MAXV, AX

    MOV AH, 4CH
    INT 21H
MAIN ENDP

MAX_OF_3 PROC
    ; Input : AX, BX, CX = three numbers
    ; Output: AX = the largest of the three
    CMP AX, BX
    JGE CHECK_C
    MOV AX, BX
CHECK_C:
    CMP AX, CX
    JGE M3_DONE
    MOV AX, CX
M3_DONE:
    RET
MAX_OF_3 ENDP

END MAIN
```

Listing 25: Maximum of three numbers procedure

5.4 Printing all primes up to N

Same trial-division logic as the macro version, just rewritten as a proper procedure this time.

```
.MODEL SMALL
.STACK 100H
.DATA
    LIMIT DW 30
```

```
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    MOV AX, LIMIT        ; parameter: AX = upper limit
    CALL PRINT_PRIMES

    MOV AH, 4CH
    INT 21H
MAIN ENDP

PRINT_PRIMES PROC
    ; Input: AX = upper limit
    PUSH AX
    PUSH BX
    PUSH CX
    PUSH DX

    MOV BX, AX           ; save limit
    MOV AX, 2            ; first candidate
OUTER_LOOP:
    CMP AX, BX
    JG PP_DONE

    PUSH AX              ; save candidate
    MOV CX, 2            ; trial divisor
INNER_LOOP:
    CMP CX, AX
    JGE IS_PRIME

    PUSH AX
    MOV DX, 0
    DIV CX
    CMP DX, 0
    POP AX
    JE NOT_PRIME
    INC CX
    JMP INNER_LOOP

IS_PRIME:
    CALL PRINT_NUM
    MOV DL, ' '
    MOV AH, 2
    INT 21H

NOT_PRIME:
    POP AX               ; restore candidate
    INC AX
    JMP OUTER_LOOP

```

```
PP_DONE:
    POP DX
    POP CX
    POP BX
    POP AX
    RET
PRINT_PRIMES ENDP

; -- include PRINT_NUM procedure here (see Section 1) --

END MAIN
```

Listing 26: Prime printing procedure

5.5 Checking if a value exists in an array

```
.MODEL SMALL
.STACK 100H
.DATA
    ARR     DB 4, 8, 15, 16, 23, 42
    LEN     EQU 6
    TARGET  DB 23
    MSG_YES DB 0DH, 0AH, 'Value found$'
    MSG_NO  DB 0DH, 0AH, 'Value not found$'
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA SI, ARR          ; parameter: SI = array address
    MOV CX, LEN          ; parameter: CX = array length
    MOV DL, TARGET        ; parameter: DL = value to search for
    CALL FIND_VALUE       ; returns AL = 1 if found, 0 if not

    CMP AL, 1
    JE PRINT_YES
    LEA DX, MSG_NO
    JMP SHOW
PRINT_YES:
    LEA DX, MSG_YES
SHOW:
    MOV AH, 9
    INT 21H

    MOV AH, 4CH
    INT 21H
MAIN ENDP

FIND_VALUE PROC
    ; Input : SI = array address, CX = length, DL = target value
    ; Output: AL = 1 if found, 0 if not found
```

```
36      PUSH CX
37      PUSH SI
38      PUSH DX
39
40      MOV AL, 0            ; assume not found
41  FV_LOOP:
42      CMP BYTE PTR [SI], DL
43      JNE FV_NEXT
44      MOV AL, 1
45      JMP FV_DONE
46  FV_NEXT:
47      INC SI
48      LOOP FV_LOOP
49
50  FV_DONE:
51      POP DX
52      POP SI
53      POP CX
54      RET
55  FIND_VALUE ENDP
56
57  END MAIN
```

Listing 27: Array search procedure

6 Things worth remembering

6.1 Array declaration and addressing

- A **byte array** (`ARR DB 10 DUP(?)`) has 1-byte elements, so the pointer moves with `INC SI`.
- A **word array** (`ARR DW 10 DUP(?)`) has 2-byte elements, so the pointer moves with `ADD SI, 2`.
- `[SI]`, `[DI]`, and `[BX]` are all fine for indexed or indirect addressing.
- `BYTE PTR` / `WORD PTR` are needed when the assembler can't figure out the size on its own – usually when comparing against an immediate value.

6.2 Stack notes

- `PUSH` always moves a full word and decrements `SP` by 2; `POP` increments `SP` by 2.
- The stack grows *downward* in memory.
- Every `PUSH` needs a matching `POP` – getting this unbalanced corrupts the return address and crashes the program.
- Pushing values in order and popping them back out is a quick way to reverse a sequence.

6.3 Macro vs. procedure

Aspect	Macro	Procedure
Defined with	`MACRO / ENDM`	`PROC / ENDP`
Invoked with	name (expanded inline)	`CALL`
Code size	grows with each use	exists once
Speed	faster (no call/ret)	slightly slower (call/ret overhead)
Local labels	need `LOCAL` keyword	labels are naturally unique

6.4 Passing parameters

- In **macros**, parameters are just substituted as text at expansion time – registers, memory operands, or literal values all work.
- In **procedures**, parameters are usually passed through registers (`AX`, `BX`, `CX`, `DX`, `SI`, `DI`) for small programs like these, or pushed onto the stack when there are more of them.
- Always `PUSH` and `POP` any register a procedure touches internally but isn't using to return a result – otherwise it silently corrupts the caller's data.

6.5 Mistakes I've made (so you don't have to)

- `LOOP` uses `CX` automatically – if the macro or procedure body also needs `CX` for something else, save and restore it, or just leave `CX` alone for counting.
- Off-by-one errors when computing the "last element" address – remember index 9 is the last one in a 10-element, 0-indexed array.
- Forgetting `LOCAL` for labels inside a macro that gets invoked more than once – causes "label already defined" errors.
- Forgetting to zero `DX` (`MOV DX, 0`) before a `DIV` on a word-sized dividend.